Learning When a Robot Should Ask For Help: Adaptive Intervention Requests In Human-Robot Teaming

Removed for Review

***Abstract*— Robots operating in complex environments occasionally encounter situations where autonomous navigation fails. Requesting and receiving a timely intervention from their human supervisor can be critical for failure prevention or recovery. However, requesting help too frequently or at inappropriate times can increase operator workload and reduce team efficiency, while delayed requests may lead to prolonged downtime or unrecoverable failures. In addition to alert timing, the human, robot and task context also influences the intervention outcomes. This paper proposes an algorithm for learning to generate adaptive human intervention requests during human-robot teaming. A help-seeking policy is trained via offline reinforcement learning based on past intervention helpfulness and autonomous progress. This learned alert policy enables a robot to selectively request human help when the intervention outcome is likely to be beneficial. We evaluate the proposed policy in different task environments and user expertise levels in simulation, followed by evaluation in a physical user study. Our findings demonstrate that the learned policy improves exploration efficiency, reduces unnecessary alerts, and enhances novice user support without compromising expert performance in real-time human-robot teaming missions.**

I. Introduction

Robots operating in complex, dynamic environments such as disaster zones, remote exploration sites, or industrial settings, often encounter situations that challenge their autonomy. For example, a robot navigating a tunnel may face unexpected debris, degraded sensing, or planner error that prevent continued progress [1], [2]. In such cases, human assistance can play an important role by providing corrective navigation guidance, such as suggesting alternative routes, placing waypoints to bypass obstacles, or helping the robot recover from localization or planning failures [3], [4], [5].

Human operators with different levels of experience may vary both in their ability to decide when to intervene and in the quality of the interventions they provide [6], [7], [8]. Even expert operators may not always choose optimal timing or strategies for assisting a robot, while novice users may lack sufficient understanding of the robot's internal state or capabilities to guide it effectively. Prior research has also identified occasions where human intervention was disadvantageous or unnecessary [3]. Users also differ in their preferences and responsiveness due to personal or contextual factors. Excessive or poorly timed requests can frustrate operators and disrupt their primary tasks or result in alert fatigue [9], while infrequent requests may lead to a loss in situational awareness [10], inefficiency or failure to recover from difficult situations. Balancing these trade-offs is essential for effective human–robot collaboration [7].

How can a robot decide when to seek help? In many practical systems, alerts are triggered based on static threshold-based rules (e.g., speed below a certain limit for a duration) to detect problematic states. While simple and interpretable, these methods often fail to account for contextual factors such as environmental difficulty, the robot's autonomous capabilities, or the effectiveness of past human interventions, which may vary across users and situations [11].

We propose an adaptive alert policy trained via offline reinforcement learning (RL), which learns to predict when a robot should request help based on historical data and current task context. Our method, illustrated in Fig. 1, leverages two key signals: (1) intervention helpfulness, characterized by the exploration progress observed following an intervention, and (2) autonomy score, representing the robot's ability to make progress independently. By optimizing alerts for scenarios where human help is likely to be beneficial and the robot is unable to progress autonomously, our approach seeks to improve efficiency while reducing unnecessary interruptions—particularly important for novice users or high-tempo missions.

We conduct a simulated experiment and a physical user study comparing our learned alert policy against a threshold-based, score-based, and LinUCB baselines across diverse environments and operator expertise levels[1]. Participants in the physical user study performed three exploration missions with different alert conditions, allowing us to assess the impact of alert strategy on exploration success, intervention dynamics, and team performance. Our contributions are:

- We propose a learning-based help-seeking policy that enables robots to decide when to request human assistance during exploration, trained via offline RL.
- We introduce a multi-head model architecture that jointly learns alert decisions and auxiliary predictions of autonomy and intervention helpfulness. These auxiliary supervised tasks provide additional learning signals that improve the policy's ability to estimate recovery capability and intervention utility.
- We evaluate the proposed method in simulated and real-world experiments, demonstrating improved exploration performance, higher progress per alert, and reduced operator workload compared to a threshold-based baseline.

II. Related Work

This section reviews prior work across three interconnected themes that motivate our approach: how robots adapt

[1]Implementation details, supplementary results, and videos are available on the project page: `https://anonymous.4open.science/r/Learning-When-a-Robot-Should-Ask-for-Help-778D/index.html`

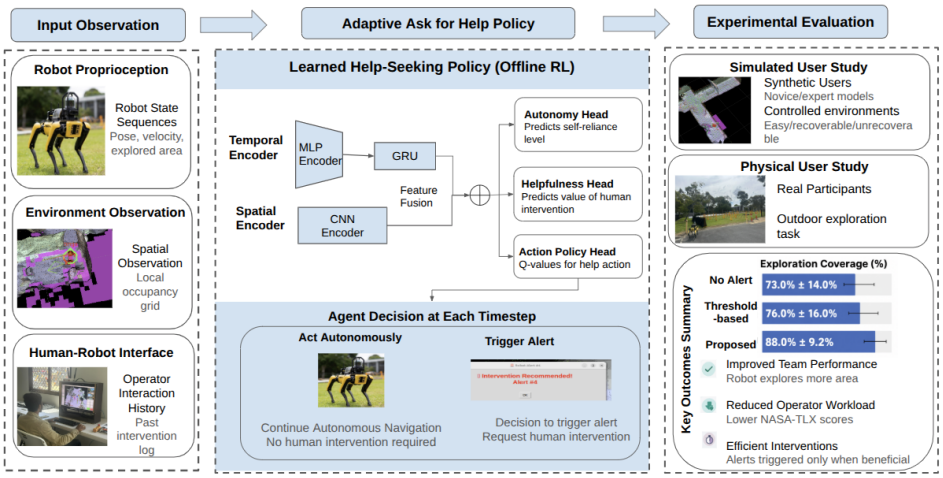

Fig. 1: The proposed reinforcement learning framework for learning a help-seeking alert policy. Our model learns when to issue alerts based on environmental context, robot autonomy and human intervention utility, resulting in efficient alerts that improve task performance while reducing operator workload, as shown in simulated and physical user studies.

behavior to different users in shared autonomy settings, how they handle the variability of human input, and how they decide when to request assistance.

Shared Autonomy and User-Adaptive Robot Behavior: In shared autonomy, robots often operate alongside human partners, relying on timely assistance to resolve failures or improve task performance [12], [7]. Prior work has shown that adapting robot behavior to different users can improve collaboration and task performance, particularly in complex or uncertain environments [13]. A common approach is to model operator expertise [14] or preferences based on interaction history or task performance [15], and to adjust autonomy levels accordingly [16].

Variability of Human Input: Human guidance may be delayed, inconsistent, or suboptimal due to limited situational awareness, cognitive load, or misunderstanding of robot capabilities [17], [18]. Prior approaches therefore estimate trust in feedback [18], weigh advice based on outcomes [19], or learn whether to accept or reject human input [20], [17]. However, these methods primarily assess input after it is provided rather than when it should be requested.

When to Request Human Assistance: More recent work has explicitly examined when robots should request human assistance during task execution, recognizing that unnecessary or poorly timed requests can be costly. Chung et al. [4] used robot introspection and policy expertise to trigger help requests when expected performance degrades. While this shows robots can autonomously identify when assistance may be needed, it assumes human input is uniformly beneficial and does not account for variation in intervention quality across task states or users. Other approaches focus on the robot's capabilities. Xie et al. [21] requested assistance in potentially irreversible states, while Igbinedion and Karaman [22] used policy uncertainty to determine when help is needed. These approaches primarily base help-seeking on the robot's estimated competence, uncertainty, or risk. Contextual bandit approaches, such as LinUCB, have also been used to learn when to request assistance from the observed context [23]. Complementary work models human factors such as helpfulness and busyness [7], while operator behavior and intervention strategies have been shown to vary with expertise [8].

Summary and Gap: Overall, existing work suggests that effective human–robot collaboration depends not only on the availability of human input, but also on when and under what circumstances it is requested. However, these help-seeking approaches [4], [21], [22] do not explicitly account for how intervention effectiveness, which may be inconsistent or suboptimal [17], [18], varies across task states or users. In contrast, our work focuses on learning help-seeking behavior that accounts for the state-dependent utility of human intervention as well as the robot's autonomous capabilities.

III. Methodology

In this work, we consider a robot exploring a challenging, partially unknown environment using semi-autonomous perception and navigation. Due to sensing uncertainty and environmental complexity, the robot may fail (e.g., get stuck or make inefficient progress). A remote human operator supervises the mission and can intervene by providing waypoint guidance, but continuous or poorly timed interventions increase workload without improving performance.

We aim to enable the robot to adaptively decide when to request human assistance, such that interventions are triggered only when likely to be beneficial. We formulate this as a decision-making problem and learn a policy that predicts the usefulness of intervention from the robot's state and past interaction outcomes. The remainder of this section presents the system formulation and learning approach.

A. Problem Formulation

We formulate robot help-seeking as a Partially Observable Markov Decision Process (POMDP) defined by the tuple

$(S, A, T, R, \gamma, \Omega, O)$. At timestep t, the environment is characterized by an underlying state $s_t \in S$, while the robot receives only a partial observation $o_t \in \Omega$ and selects an action $a_t \in A = \{0, 1\}$. Here, $a_t = 0$ denotes continuing autonomous exploration and $a_t = 1$ denotes requesting human assistance.

The underlying state captures three categories of information relevant to the help-seeking decision: (i) the robot's navigation condition, (ii) the local environmental condition, and (iii) latent factors determining the utility of requesting assistance in the current context. These latent factors include whether human assistance is likely to improve task progress and the human operator's expertise. The observation function $O(o_t \mid s_t)$ specifies the conditional distribution of observations given the underlying state, where o_t contains robot motion, exploration progress, local costmap information, and observable interaction history.

To capture temporal dependencies, each observation is represented using a sliding window of the past K timesteps, i.e., $\mathcal{H}_t = (o_{t-K+1}, \ldots, o_t)$. This allows the policy to reason over trends in robot progress, such as stagnation or recovery. Rather than explicitly estimating the POMDP belief state, we learn an implicit belief representation: $z_t = f_\theta(\mathcal{H}_t), z_t \in \mathbb{R}^d$, where f_θ combines recurrent encoding of the scalar observation history with convolutional encoding of the local costmap (Sec. III-E).

To encourage z_t to encode task-relevant outcome information, we introduce two auxiliary prediction objectives: an autonomous-progress head $\hat{A}_t = g_A(z_t)$ estimating the expected outcome under continued autonomous operation, and a helpfulness head $\hat{H}_t = g_H(z_t)$ estimating the expected outcome associated with human assistance. Both are supervised using retrospective targets from the logged trajectories, providing task-relevant supervision to the shared representation; the alert action itself is determined by the learned Q-values.

The transition function $T(s_{t+1} \mid s_t, a_t, u_t)$ depends on both robot control and possible human intervention. If the robot triggers an alert (a_t=1), the operator may provide an intervention u_t (e.g., a corrective waypoint), which influences the subsequent trajectory and next state s_{t+1}. The reward function $R(s_t, a_t, u_t)$ rewards exploration progress, penalizes ineffective or unnecessary alerts, and rewards timely interventions that improve performance. We aim to learn a help-seeking policy $\pi(a_t \mid z_t)$ that maximizes the expected return over a finite-horizon episode of length T, using n-step temporal difference learning with discount factor $\gamma \in [0, 1]$:

$$Q(z_t, a_t) \approx \sum_{i=0}^{n-1} \gamma^i r_{t+i} + \gamma^n \max_{a'} Q(z_{t+n}, a').$$

Here, r_{t+i} is the observed scalar reward at step $t + i$, γ discounts future rewards to reflect temporal preference, and n controls the bootstrapping horizon. The Q-value is bootstrapped from the predicted value of the state at time $t+n$, using the maximum value over possible next actions a'. To enable stable policy learning from offline data, we adopt a *Conservative Q-Learning (CQL)* framework (see Sec. III-E), which regularizes Q-values by penalizing overestimation on out-of-distribution actions not well represented in the dataset.

B. Overview and Policy Architecture

We adopt an offline RL framework that trains a binary alert policy from logged robot behavior, intervention events, and mission outcomes. The policy network takes as input a sequence of scalar features (e.g., velocity, pose change, explored area, time since last alert) and a stack of local costmaps, enabling it to identify temporal patterns such as gradual stagnation or delayed recovery.

As shown in Fig. 1, scalar inputs are encoded by an MLP followed by a GRU, while costmaps are encoded by a CNN. The resulting features are fused into a shared representation and passed to three parallel heads. Each head uses a fully connected hidden layer with ReLU activation. The Q-value head outputs two action values, while the autonomous-progress and helpfulness heads each produce a single sigmoid output.

C. Auxiliary Predictive Models

To provide additional context for alert decisions, we introduce two auxiliary heads: a human helpfulness predictor and a robot autonomy predictor. Both are trained via supervised learning on logged mission data. Their predictions serve as auxiliary learning targets that regularize the shared representation and help the policy reason about the expected outcomes of either requesting help or continuing autonomous operation.

1) Human Helpfulness Predictor: This module estimates the observed outcome associated with a human intervention given the current context (value range [0,1]). Rather than modeling operator willingness [7], we ground prediction in the task-level outcomes: ground-truth labels are derived from post-intervention changes in explored area, where substantial coverage gains indicate a helpful intervention, and minimal progress indicates otherwise. The model learns to predict whether an intervention in the current context is likely to be followed by favorable exploration progress, based on the operator's intervention history and the robot's local state.

2) Robot Autonomy Predictor: This module estimates the robot's ability to make progress without assistance (value range [0,1]). Ground-truth scores are computed from autonomous operation intervals, excluding periods in which the robot executes user-provided waypoints after human intervention, to isolate its inherent exploration capability. The predictor learns to estimate this autonomous progress rate from local state and environmental observations, enabling the policy to anticipate whether continued autonomy is likely to be productive or not.

D. Reward Function

To guide the learning of the help-seeking policy, we design a reward function $R(s_t, a_t, u_t)$ that reflects both per-step exploration performance and the impact of human interventions. It captures the following key components:

- **Exploration progress:** At each timestep t, the change in explored area is computed relative to the previous step. A reward of $+1$ is assigned if progress is observed, and -1 otherwise.
- **Alert–intervention outcomes:** When the robot triggers an alert ($a_t = 1$) and a human intervention u_t follows, we compute the improvement in explored area before and after the intervention. If the intervention leads to measurable progress, a shaped reward is given:

$$r_t = 1 + 10 \cdot \left(\frac{1}{1 + e^{-\Delta\text{area}}} \right),$$

  where Δarea is the increase in the explored area post-intervention. If no progress occurs, a penalty of -1 is applied.
- **No intervention:** If an alert is triggered and no help is received, but the robot makes progress, a small penalty (-1) is applied to reflect unnecessary alerting. Conversely, if the robot remains stuck, a reward of $+1$ is assigned, indicating that the alert was appropriately triggered despite the absence of intervention.

This reward formulation encourages exploration progress while incentivizing necessary alerts that lead to effective human assistance.

E. Implementation and Training Details

The proposed policy is implemented as a deep Q-network with auxiliary prediction heads.

We train the model offline using robot-human interaction data collected from 134 semi-autonomous navigation episodes in a high-fidelity simulated robot exploration environment. The dataset was originally introduced in [8] and contains intervention data collected during exploration tasks in unstructured tunnel-like environments. The system supports perception, mapping, and planning, enabling exploration in unstructured, tunnel-like environments. Human operators can intervene by providing waypoint guidance when the robot fails to make progress.

The dataset [8] includes four representative scenarios with annotated challenging regions, capturing common navigation difficulties such as narrow passages, intersections, dead ends, and uneven terrain. These scenarios induce a range of failure modes, including stagnation and obstacle-induced immobilization, requiring human intervention. To increase behavioral diversity, we further augment human waypoint-intervention data using the expertise-conditioned user model described in Section III-G.

Each episode includes time-aligned state logs, costmaps, simulated human interventions, and mission outcomes. The final dataset contains over 99,000 training sequences, each consisting of 100 timesteps. These sequences are used to populate a replay buffer, from which fixed-length samples are drawn during training. Each sample includes the current and next state sequences, costmaps, actions, scalar rewards, and auxiliary supervision signals.

The training objective jointly optimizes the Q-values and auxiliary scores:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{Q}} + \lambda_{\text{aux}} \left(\mathcal{L}_{\text{help}} + \mathcal{L}_{\text{auto}} \right) + \lambda_{\text{CQL}} \cdot \mathcal{L}_{\text{CQL}},$$

where $\mathcal{L}_{\text{Q}}$ is a mean-squared Bellman loss using n-step temporal difference targets, $\mathcal{L}_{\text{help}}$ and $\mathcal{L}_{\text{auto}}$ are auxiliary prediction losses (L_2 loss), and $\mathcal{L}_{\text{CQL}}$ is a regularization term from CQL that penalizes overestimated Q-values for out-of-distribution actions. All losses are computed jointly, and no network component is pre-trained or frozen. In this work, $\lambda_{\text{aux}} = 0.5$, $\lambda_{\text{CQL}} = 1$.

We use the Adam optimizer with a learning rate of 10^{-4}, a batch size of 64, and gradient norm clipping at 1.0. Target networks are updated every 10 epochs via hard synchronization. Scalar inputs are standardized using global statistics, and sigmoid normalization is applied to helpfulness and autonomy scores. The trained model was saved after offline training and deployed for inference on a local computer during evaluation.

F. Baseline Policies

1) Threshold-Based Alert Policy: A rule-based system monitors robot motion and explored area. A stuck alert is raised when displacement stays below 0.04 m for more than 12 s; an inefficiency alert is raised when the explored area does not increase over a 10 s window, indicating repeated traversal or slow progress despite continued motion. All thresholds were tuned by grid search on validation episodes disjoint from the evaluation maps. Alerts use a generic message prompting operator intervention.

2) Score-Based Alert Policy: This baseline uses the auxiliary predictions without the RL Q-value head. It requests assistance when predicted autonomous progress is below τ_A and predicted intervention outcome is above τ_H, with validation-selected thresholds $\tau_A = \tau_H = 0.2$. Unlike the proposed policy, it does not optimize sequential return.

3) LinUCB Contextual-Bandit Policy: We adapt LinUCB [23] to the binary help-seeking decision using the same offline data, reward, and observable features as the proposed policy. The action maximising the LinUCB reward estimate and uncertainty bonus is selected. Validation gives $\alpha = 0.5$ and ridge coefficient $\lambda = 1.0$. Parameters remain fixed during evaluation, with no online updates.

G. User Expertise Model

To simulate human interventions with varying expertise levels, we learn a data-driven model that predicts intervention waypoints from robot state and local environment context. The model takes as input the robot's pose, velocity, local costmap features, and derived autonomy indicators, and outputs a continuous navigation target representing a human-provided intervention.

The model is trained on a user study dataset consisting of 27 participants, including both novice and expert operators [8]. Intermediate participants are retained and assigned to the novice or expert group according to their task performance in that study, and a separate regressor is trained for each group. The dataset captures human intervention behavior in

robot exploration tasks, including waypoint corrections under different navigation conditions. To improve generalization given the limited dataset size, we apply data augmentation by perturbing intervention timing and robot state features.

Each regressor is implemented as a gradient-boosted multi-output regressor with 100 estimators, a learning rate of 0.05, a maximum tree depth of 3, and squared-error loss, achieving low prediction error (MAE = 0.063, MSE = 0.071). Because the prior study found that robot-to-waypoint intervention distance differs with operator expertise [8], we additionally construct an empirical distance distribution for each group. For each rollout, a user-specific intervention distance is sampled from the corresponding distribution and held fixed for the rollout duration, so that rollouts within a group vary in intervention behavior while preserving the group-level differences observed in the real-user data.

IV. Evaluation

We evaluate a learned alert policy for robot navigation using offline simulations (controlled benchmarking using simulated robot and user data) and an online physical user study (real-time evaluation with novice users in a human-robot team exploration task).

A. Hypotheses

- H1: The learned alert policy improves team performance compared to the threshold-based alert policy.
- H2: The learned alert policy reduces operator workload during robot supervision tasks.

B. Offline Simulation Evaluation with Simulated Users

We first conducted an offline evaluation in a simulator to examine the behavior of the learned alert policy under controlled conditions. This simulation environment involves a human-robot team exploration task similar to the physical study (Section IV-C) and the user expertise model training dataset (Section III-G), in which the simulated user provides waypoint intervention when alerted by the robot. The simulation allows us to reduce variability in human response while preserving variation in intervention quality.

In this setting, simulated users respond immediately to the robot's alerts, removing variability in response timing. Differences in intervention quality are retained by modeling users with different expertise levels (novice and expert). Intervention waypoints are sampled from distributions of the learned user model (Section III-G), allowing the simulation to represent both helpful and less helpful interventions. A total of 100 simulated interaction rollouts (50 novice-derived and 50 expert-derived rollouts) are generated to support comparisons across alert policies.

Evaluation is performed on a set of fixed maps that differ from those used during training or online physical user studies (details in supplementary material). Variations in map structure lead to different failure locations and intervention timings, resulting in diverse interaction trajectories.

Alerts are generated by the proposed policy, baseline policies described in Section III-F, or ablated policy variants. The ablation removes the autonomy and/or helpfulness auxiliary objectives to assess their individual and combined contributions. These methods are evaluated under consistent conditions across three environment-difficulty levels and novice- and expert-derived intervention conditions. We additionally conducted a paired counterfactual analysis to assess whether post-intervention progress reflects a benefit of human assistance beyond autonomous recovery. We randomly selected 40 robot states and replayed each state with human intervention and autonomous continuation under closely matched initial conditions.

C. Physical User Study

The study received approval from the institutional ethics committee. Twenty participants were recruited through invitations circulated within a research institution and a university, along with snowball sampling. Prior robot-supervision and navigation-based video-game experience was also recorded using four frequency levels (never to frequently). Participants first completed a training session supervising a Boston Dynamics Spot robot to explore an outdoor environment. The robot runs a field-tested subterranean-exploration autonomy stack [1], comprising multi-agent 3D SLAM, multiple RGB and thermal cameras, a spinning lidar, a global frontier-based exploration planner, and a search-based local planner with obstacle avoidance and replanning. Prior to the user study, we conducted fully autonomous runs without human intervention to characterize where this stack degraded in the test environment.The physical environment was not designed to replicate the simulation geometry, but contained analogous robot-level challenges, including constrained passages, obstacles, navigation deadlocks, and exploration oscillations; representative physical failure cases are shown in the supplementary video.

Each participant completed three 10-minute exploration sessions under different alerting conditions: no alerts, threshold-based alerts, and adaptive alerts generated by the learned policy. The order of conditions was randomized across participants, with short breaks and brief questionnaires administered between sessions. Participants were instructed to supervise the robot naturally without prescribed strategies.

During each session, participants performed three concurrent tasks. They monitored the robot's autonomous exploration using a map-based interface, responded to alerts when present by deciding whether or not an intervention was necessary, and placed waypoints to guide recovery if assistance was deemed necessary. The robot followed the participant's waypoint and then resumed autonomous exploration. In parallel, participants verified robot-detected objects by reviewing RGB images and associated labels, accepting or rejecting detections via keyboard input. Object verification occurred continuously throughout each session, introducing supervisory workload alongside navigation assistance.

D. Measures

We evaluate alert policies using both objective performance metrics and subjective user assessments. The follow-

ing objective metrics are used:

- **Total explored area**: The cumulative area covered by the robot during the mission, representing overall task performance.
- **Recovery time**: The duration between an alert and the robot's resumption of effective exploration. In the simulation study, where user response timing is fixed, this metric primarily reflects intervention effectiveness. In the physical user study, it reflects both human response time and intervention effectiveness.
- **Progress per alert**: Total explored area divided by the number of alerts, measuring mission-level alert efficiency rather than the causal effect of individual alerts
- **Post-alert progress**: The increase in explored area within a fixed horizon (T=10s) following an alert, capturing immediate post-alert progress while limiting unrelated later autonomous progress.
- **Alert success rate (physical user study only)**: The proportion of alerts that lead to a human intervention and measurable post-intervention exploration progress before the next alert or the end of the session.

In the physical user study, subjective measures were also collected to assess operator experience. After each trial, participants completed the NASA Task Load Index (NASA-TLX) [24] to measure perceived workload. Trust in the robot was assessed using Muir's trust questionnaire [25]. Participants also rated the perceived usefulness of alerts and completed a short qualitative questionnaire describing their preferred alert strategy and suggestions for improving alert timing or design.

V. Results

A. Offline Simulation Results

Results are summarized in Table I under expert- and novice-derived intervention conditions, aggregated across repeated trials. Statistical comparisons were conducted using Welch's two-sample t-tests, with 95% confidence intervals and Cohen's d effect sizes.

The proposed policy achieved the best overall performance across both conditions. Compared with the threshold-based, score-based, and LinUCB baselines, it achieved larger explored area ("Area"), shorter recovery time ("Rec. (s)"), and higher progress per alert ("Prog./Alert") and post-alert progress. Improvements in explored area and post-alert progress were significant against all baselines ($p < 0.001$, $d = 0.91$–2.85). The ablation results further show that the full model significantly improved explored area and post-alert progress over RL only under both conditions ($p < 0.001$, $d = 1.22$–2.12). The individual auxiliary objectives provided complementary and condition-dependent benefits, while their combination yielded the most consistent overall performance. The approximate paired counterfactual analysis further showed 7.7 m^2 greater exploration progress following intervention than autonomous continuation under closely matched conditions (95% CI: $[6.5, 9.0]$, $p < 0.001$).

TABLE I: Offline simulation performance and auxiliary-objective ablation under expert- and novice-derived intervention conditions (mean ± s.d.).

User	Policy	Area (m^2)	Rec. (s)	Prog./Alert (m^2/alert)	Post-Ale. Prog. (m^2)
Exp.	Threshold	199.0±21.0	70.5±19.3	8.3±2.2	7.1±2.4
	Score	218.6±12.7	43.8±13.5	14.6±2.4	9.4±2.2
	LinUCB	224.3±10.9	38.7±11.2	16.8±2.1	9.9±2.1
	RL only	220.2±12.9	40.4±11.8	17.4±1.5	10.5±2.1
	RL+A	224.5±10.7	37.1±10.2	18.1±2.1	11.2±3.3
	RL+H	226.7±8.9	35.2±9.1	18.8±1.2	12.0±5.1
	Proposed	**232.0±4.5**	**29.1±10.4**	**21.1±2.5**	**15.0±4.1**
Nov.	Threshold	189.7±37.3	100.6±27.9	4.7±1.9	5.3±2.2
	Score	205.4±14.8	63.5±16.2	10.8±1.7	6.9±2.1
	LinUCB	205.8±13.1	56.7±11.9	13.2±1.5	8.0±2.0
	RL only	198.6±15.8	45.7±10.4	14.8±1.6	6.7±2.5
	RL+A	215.9±13.6	42.5±5.7	15.5±0.8	8.1±2.4
	RL+H	203.8±12.1	43.9±6.8	15.2±1.1	9.5±3.3
	Proposed	**217.0±10.4**	**41.2±3.2**	**16.7±0.1**	**12.0±2.5**

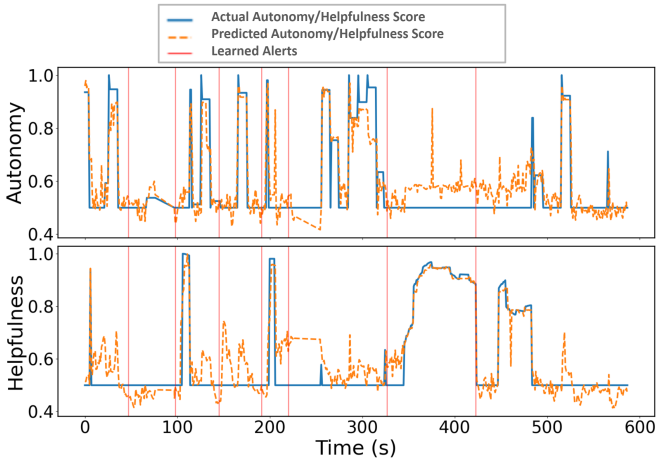

Fig. 2: Predicted vs. Actual helpfulness and autonomy scores and the learned alert timings in an example user study trial.

Overall, these results support H1 and demonstrate improved exploration performance and alert efficiency across intervention conditions.

B. Physical User Study Results

1) Prediction Accuracy and Participant Adaptation: Across all participants, the auxiliary prediction heads achieve low mean squared error (MSE) for helpfulness (0.0091 ± 0.0090) and autonomy (0.0072 ± 0.0052), indicating close alignment with target signals. Predicted helpfulness also matches the distribution of ground-truth values across participants (GT: 0.65 ± 0.08, Pred: 0.63 ± 0.07), suggesting that the model captures variability in intervention effectiveness across users.

Figure 2 shows the temporal behavior of the predictions in a representative trial. The model tracks the overall trends of both helpfulness and autonomy, with predicted signals closely following the ground truth. Alert events are generally aligned with transitions in these signals, suggesting that the

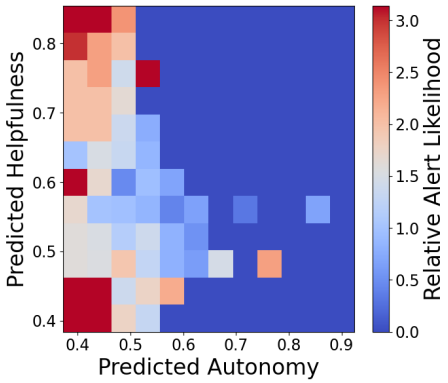

Fig. 3: Visualization of the learned alert policy over the normalised autonomy–helpfulness state space. The space is discretized into a grid, and each cell shows the relative alert likelihood normalized by the overall alert rate. Values greater than one indicate higher-than-average alert frequency. Sparsely populated regions are smoothed using nearest-neighbor interpolation.

learned policy responds to changes in robot autonomy and intervention effectiveness, despite the policy being trained in a simulated environment.

2) Objective Measures: Table II summarizes the user study results. The proposed adaptive alert policy achieved the highest explored area, significantly outperforming both threshold and no-alert baselines, supporting H1. It also generated fewer alerts than the threshold condition, indicating reduced operator interruptions while improving performance. Regarding alert efficiency, the learned policy achieved higher progress per alert and alert success rate, suggesting more effective intervention timing. Recovery time showed a descriptive improvement but was not statistically significant.

3) Subjective Measures: As summarizes in Table II, perceived workload differs significantly across conditions, with the learned alert policy yielding the lowest NASA-TLX scores, followed by the threshold and no-alert conditions, supporting H2. In contrast, trust and perceived alert usefulness do not show significant differences across conditions.

Qualitative responses indicated that participants experienced difficulty dividing attention between robot supervision and the concurrent object classification task. Monitoring multiple interface panels increased cognitive load, and participants reported that alerts reduced the need for continuous manual monitoring, particularly during periods of high task demand. Participants primarily relied on observable robot behaviours, such as becoming stationary, slowing down, or revisiting explored areas, as primary cues for intervention. Participants noted that alerts were most beneficial when their attention was occupied by the secondary task, as alerts acted as reminders to re-engage with the robot (e.g., "Great to have assistance while I was verifying objects" [P001]). These observations align with the quantitative results, where the learned alert policy reduced workload and the number of alerts while improving overall task performance.

4) Understanding When the Robot Requests Help: Figure 3 shows the alert likelihood over the autonomy–helpfulness state space. Alerts are concentrated in regions of low predicted autonomy (left-side), indicating that the policy requests assistance when autonomous recovery is unlikely, while avoiding unnecessary alerts in high-autonomy states (right-side). Within low-autonomy regions, alert likelihood increases with predicted helpfulness (top-left), suggesting that the policy prioritizes situations where intervention is expected to be beneficial.

VI. Discussion

This work demonstrates that robots can effectively learn when to request human assistance during exploration. In simulation, the proposed policy outperformed the threshold-based, score-based, and LinUCB baselines, while the physical user study showed improved exploration performance and reduced supervisory workload.

A key factor underlying this behavior is the model's ability to estimate autonomy and intervention helpfulness. These predictions provide a structured basis for deciding when the robot is unlikely to recover independently and when human input is likely to improve progress. Their consistency across participants and alignment with robot behavior suggest that these representations generalize beyond the training setting and remain effective under real-world variability. The ablation results further show that the autonomy and helpfulness objectives provide complementary, condition-dependent benefits, with their combination yielding the most consistent overall performance. Moreover, the improvement over the score-based and LinUCB baselines suggests that the gains are not explained by the auxiliary predictions or context-dependent decision making alone, supporting the benefit of integrating these signals within a sequential RL policy.

The policy visualization further supports this interpretation. The heatmap analysis reveals that alert decisions are structured in the autonomy–helpfulness state space. Alerts are predominantly triggered in regions of low predicted autonomy, indicating that the policy identifies situations where the robot is unlikely to recover independently. Within these regions, alert likelihood increases with predicted helpfulness, suggesting that the policy further prioritizes states where human intervention is expected to be beneficial.

Unlike fixed heuristics (e.g., velocity thresholds), our adaptive approach reduces unnecessary requests, allowing operators to allocate attention effectively. Subjective results confirm a significant reduction in perceived workload in the learned alert condition. Interestingly, trust remained comparable across strategies. The slightly higher trust in threshold-based alerts likely stems from the consistent triggers that align with intuitive failure cues (e.g., stopping) that match users' existing mental models of robot behavior [26].

Several limitations should be noted. First, the simulation assumes immediate responses to alerts, whereas real users may delay or ignore alerts, introducing variability in outcomes. Second, the physical study primarily involved a relatively small and primarily novice participant sample; future

TABLE II: Summary of physical user study results (mean $\pm$ SD). Statistical significance is evaluated using Friedman tests with post-hoc Wilcoxon signed-rank tests (Bonferroni-corrected). Effect sizes are reported using r where available.

Metric	No Alert	Threshold	Proposed Adaptive Alert	p-value	Effect Size
Total explored area (m^2)	569.38 ± 108.63	596.24 ± 125.60	$\mathbf{678.33 \pm 121.90}$	$p < .001$	$W = 0.47$
Number of alerts	–	17.0 ± 6.30	$\mathbf{10.70 \pm 3.90}$	$p = .042$	$r = 0.43$
Progress per alert (m^2/alert)	–	49.81 ± 21.81	$\mathbf{73.12 \pm 35.19}$	$p = .016$	$r = 0.54$
Recovery time (s)	–	14.01 ± 11.90	11.02 ± 10.40	$p = .056$	$r = 0.43$
Alert success rate	–	0.47 ± 0.23	$\mathbf{0.63 \pm 0.19}$	$p = .048$	$r = 0.43$
NASA-TLX (workload)	51.54 ± 15.61	43.78 ± 14.21	$\mathbf{39.45 \pm 12.20}$	$p < .001$	$W = 0.53$
Trust score	3.35 ± 0.89	3.73 ± 0.98	3.48 ± 0.91	$p = .331$	$W = 0.06$
Alert usefulness	–	5.21 ± 0.55	5.29 ± 0.59	$p = .850$	$r = 0.23$

work should examine how experienced operators interact with adaptive alerting policies. Third, although the policy generalizes from simulation to real-world settings, evaluation was limited to a specific exploration task and environment.

VII. Conclusions and Future Work

This paper investigated how robots can learn when to request human assistance during exploration tasks. We proposed an RL-based alert policy that adapts help requests based on autonomous progress and past intervention outcomes. Across both simulation and real-world user studies, the learned policy improved exploration performance while issuing fewer alerts and reducing operator workload. Overall, this work highlights the importance of adaptive help-seeking for balancing autonomy and human supervision. Future work will explore user-aware models, online adaptation, additional intervention modalities, and multi-robot settings in which several robots compete for limited operator attention.

References

[1] N. Kottege *et al.*, "Heterogeneous robot teams with unified perception and autonomy: How team CSIRO Data61 tied for the top score at the DARPA subterranean challenge," *Field Robotics*, vol. 4, pp. 313–359, 2024.

[2] M. Petrlik *et al.*, "UAVs beneath the surface: Cooperative autonomy for subterranean search and rescue in DARPA SubT," *Field Robotics*, vol. 3, no. 1, pp. 1–68, 2023.

[3] N. Robinson, J. Williams, D. Howard, B. Tidd, F. Talbot, B. Wood, A. Pitt, N. Kottege, and D. Kulić, "Human-robot team performance compared to full robot autonomy in 16 real-world search and rescue missions: Adaptation of the darpa subterranean challenge," *ACM Trans. Hum.-Robot Interact.*, vol. 14, no. 1, pp. 1–30, 2024.

[4] J. J. Chung, L. Milliken, G. A. Hollinger, and K. Tumer, "When to ask for help: Introspection in multi-robot teams," in *Proc. IEEE Int. Conf. Robot. Autom. Workshops*, 2017.

[5] E. Messina, A. Jacoff, J. Scholtz, C. Schlenoff, H. Huang, A. Lytle, and J. Blitch, "Statement of requirements for urban search and rescue robot performance standards," *NIST Draft Report*, 2005.

[6] N. Conlon, D. Szafir, and N. Ahmed, ""i'm confident this will end poorly": Robot proficiency self-assessment in human-robot teaming," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 2127–2134.

[7] A. Nanavati, C. I. Mavrogiannis, K. Weatherwax, L. Takayama, M. Cakmak, and S. S. Srinivasa, "Modeling human helpfulness with individual and contextual factors for robot planning." in *Proc. Robot. Sci. Syst.*, 2021, pp. 1–11.

[8] Y. Jiang, P. Sikka, L. Tian, D. Kuliic, and C. Paris, "Influence of operator expertise on robot supervision and intervention," *arXiv:2601.15069*, 2026.

[9] S. Tariq, M. Baruwal Chhetri, S. Nepal, and C. Paris, "Alert fatigue in security operations centres: Research challenges and opportunities," *ACM Comput. Surv.*, vol. 57, no. 9, pp. 1–38, 2025.

[10] H. Senaratne, L. Tian, P. Sikka, J. Williams, D. Howard, D. Kulić, and C. Paris, "A framework for dynamic situational awareness in human–robot teams: An interview study," *ACM Trans. Hum.-Robot Interact.*, vol. 15, no. 1, pp. 1–37, 2025.

[11] S. Honig and T. Oron-Gilad, "Understanding and resolving failures in human-robot interaction: Literature review and model development," *Front. Psychol.*, vol. 9, p. 861, 2018.

[12] X. Yu, M. Hoggenmüller, and M. Tomitsch, "Encouraging bystander assistance for urban robots: Introducing playful robot help-seeking as a strategy," in *Proc. ACM Des. Interact. Syst. Conf.*, 2024, pp. 2514–2529.

[13] S. Nikolaidis, A. Kuznetsov, D. Hsu, and S. Srinivasa, "Formalizing human-robot mutual adaptation: A bounded memory model," in *Proc. ACM/IEEE Int. Conf. Hum.-Robot Interact.* IEEE, 2016, pp. 75–82.

[14] L. Milliken and G. A. Hollinger, "Modeling user expertise for choosing levels of shared autonomy," in *Proc. IEEE Int. Conf. Robot. Autom.* IEEE, 2017, pp. 2285–2291.

[15] M. Natarajan, C. Xue, S. van Waveren, K. Feigh, and M. Gombolay, "Mixed-initiative human-robot teaming under suboptimality with online bayesian adaptation," *arXiv:2403.16178*, 2024.

[16] M. K. M. Rabby, A. Karimoddini, M. A. Khan, and S. Jiang, "A learning-based adjustable autonomy framework for human–robot collaboration," *IEEE Trans. Ind. Informat.*, vol. 18, no. 9, pp. 6171–6180, 2022.

[17] M. Sridharan, "Augmented reinforcement learning for interaction with non-expert humans in agent domains," in *Proc. Int. Conf. Mach. Learn. Appl.*, vol. 1. IEEE, 2011, pp. 424–429.

[18] T. Love, R. Ajoodha, and B. Rosman, "Should i trust you? incorporating unreliable expert advice in human-agent interaction," in *Proc. Workshop Hum.-Aligned Reinforcement Learning*, 2021.

[19] R. Loftin, B. Peng, J. MacGlashan, M. L. Littman, M. E. Taylor, J. Huang, and D. L. Roberts, "Learning behaviors via human-delivered discrete feedback: modeling implicit feedback strategies to speed up learning," *Auton. Agents Multi-Agent Syst.*, vol. 30, no. 1, pp. 30–59, 2016.

[20] R. Arakawa, S. Kobayashi, Y. Unno, Y. Tsuboi, and S.-i. Maeda, "Dqn-tamer: Human-in-the-loop reinforcement learning with intractable feedback," *arXiv:1810.11748*, 2018.

[21] A. Xie, F. Tajwar, A. Sharma, and C. Finn, "When to ask for help: Proactive interventions in autonomous reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 16 918–16 930.

[22] I. Igbinedion and S. Karaman, "Learning when to ask for help: Efficient interactive navigation via implicit uncertainty estimation," in *Proc. IEEE Int. Conf. Robot. Autom.* IEEE, 2024, pp. 9593–9599.

[23] R. Banerjee, R. K. Jenamani, S. Vasudev, A. Nanavati, K. Dimitropoulou, S. Dean, and T. Bhattacharjee, "To ask or not to ask: Human-in-the-loop contextual bandits with applications in robot-assisted feeding," *arXiv:2405.06908*, 2024.

[24] S. G. Hart, "Nasa task load index (nasa-tlx): 20 years later," in *Proc. Hum. Factors Ergon. Soc. Annu. Meet.*, vol. 50, no. 9, 2006, pp. 904–908.

[25] M. Chen, S. Nikolaidis, H. Soh, D. Hsu, and S. Srinivasa, "Planning with trust for human-robot collaboration," in *Proc. ACM/IEEE Int. Conf. Hum.-Robot Interact.*, 2018, pp. 307–315.

[26] J. D. Lee and K. A. See, "Trust in automation: Designing for appropriate reliance," *Hum. Factors*, vol. 46, no. 1, pp. 50–80, 2004.